

the static and non static members which we can write inside a class ?

The following diagram explains all the **static and non static members** which we can write inside a java class.

Static Members : Loaded OR executed during the loading of the .class file

Non static Members : Executed at the time of creating the Object.

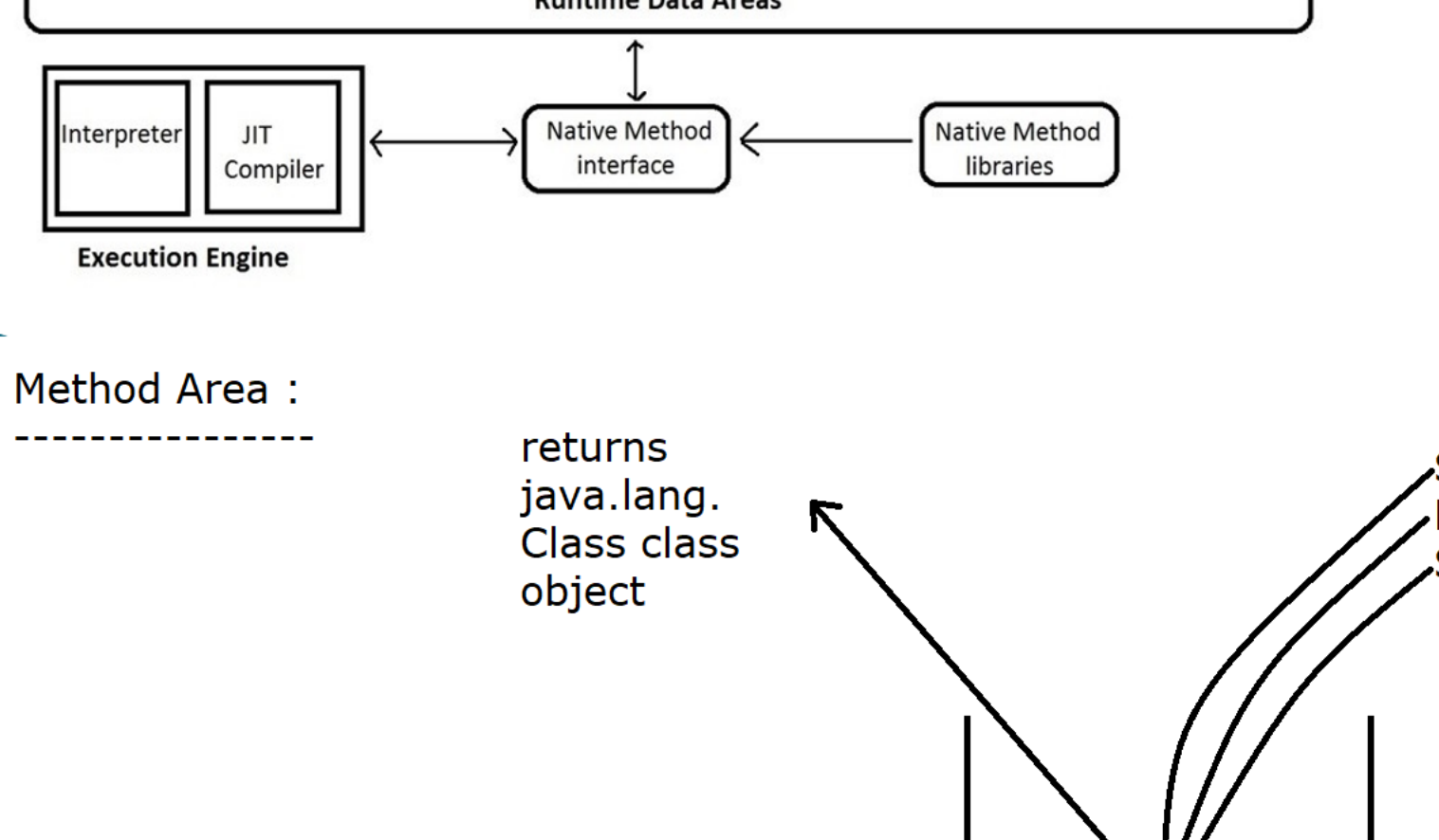
Class Members	
=====	
static member	Non static member
Class Loading Layer 1 a) Static field b) Static Block c) Static Method d) Static Nested Class	Object creation Layer2 a) Non static Field b) Non static Block c) Constructor d) Non static Method e) Non static inner class

Runtime Data Areas (Memory Areas) :

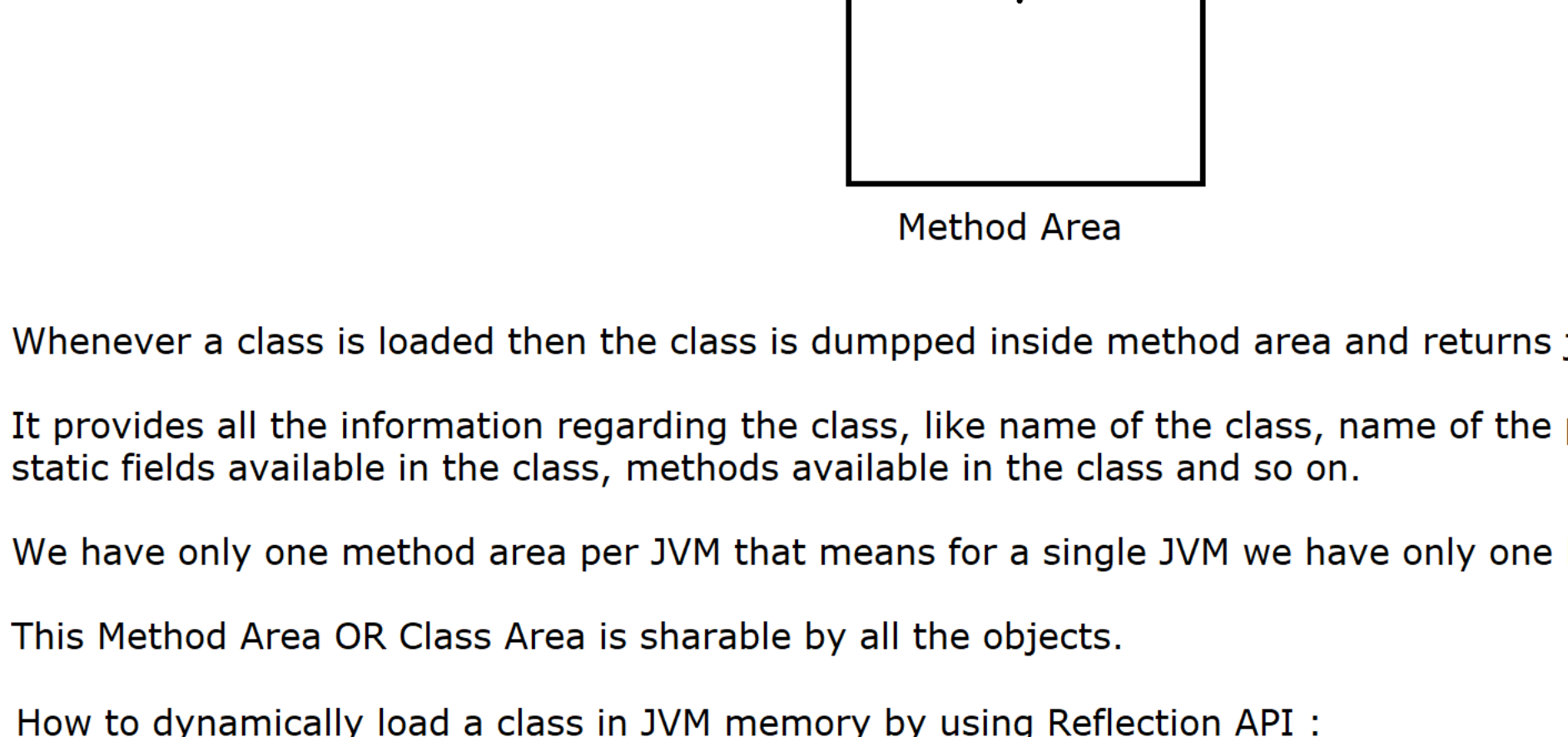
* Once the class is loaded and execution of the program is started then based on type and content different memories are created to hold appropriate data.

- 1) Method Area [To hold class related information, Only one method Area per JVM]
- 2) HEAP Area [To hold object level Data, Only one HEAP Area per JVM]
- 3) STACK Area [To hold method level Data, We have multiple Stacks for a single JVM]
- 4) PC Register [Program Counter Register]
- 5) Native Method Stack

Components in JVM Architecture



Method Area :



Whenever a class is loaded then the class is dumped inside method area and returns java.lang.Class class object.

It provides all the information regarding the class, like name of the class, name of the package, static and non static fields available in the class, methods available in the class and so on.

We have only one method area per JVM that means for a single JVM we have only one Method area.

This Method Area OR Class Area is sharable by all the objects.

How to dynamically load a class in JVM memory by using Reflection API :

* There is a predefined class called **Class available in java.lang package and this class called Class contains a predefined static method called forName(String className), this forName() method is used to load a .class file into JVM memory dynamically.**

//static factory method

public static java.lang.Class forName(String className) throws ClassNotFoundException

* This forName(String className) method throws a checked exception called ClassNotFoundException

```
class Alpha
{
    static
    {
        IO.println("SB of Alpha class");
    }
}
public class DynamicClassLoading
{
    public static void main(String[] args) throws Exception
    {
        //Reflection API
        Class.forName("Alpha"); //Alpha class is loaded
    }
}
```

* In ECLIPSE IDE, if we want to load a class then FQN (Fully Qualified Name) [Package Name + Class Name] is required as shown in the program

```
package com.ravi.class_infirmation;

class Alpha
{
    static
    {
        IO.println("SB of Alpha class");
    }
}

public class DynamicClassLoading
{
    public static void main(String[] args) throws Exception
    {
        Class.forName("com.ravi.class_infirmation.Alpha"); //Fully Qualified Name
        //Package Name + class name
    }
}
```

HEAP AREA :

Whenever we create an object in java then the properties and behavior of the object are stored in a special memory area called HEAP AREA.

We have only one HEAP AREA per JVM.

STACK Area :

All the methods are executed as a part of Stack Area.

Whenever we call a method in java then internally one stack Frame will be created to hold method related information.

Every Stack frame contains 3 parts :

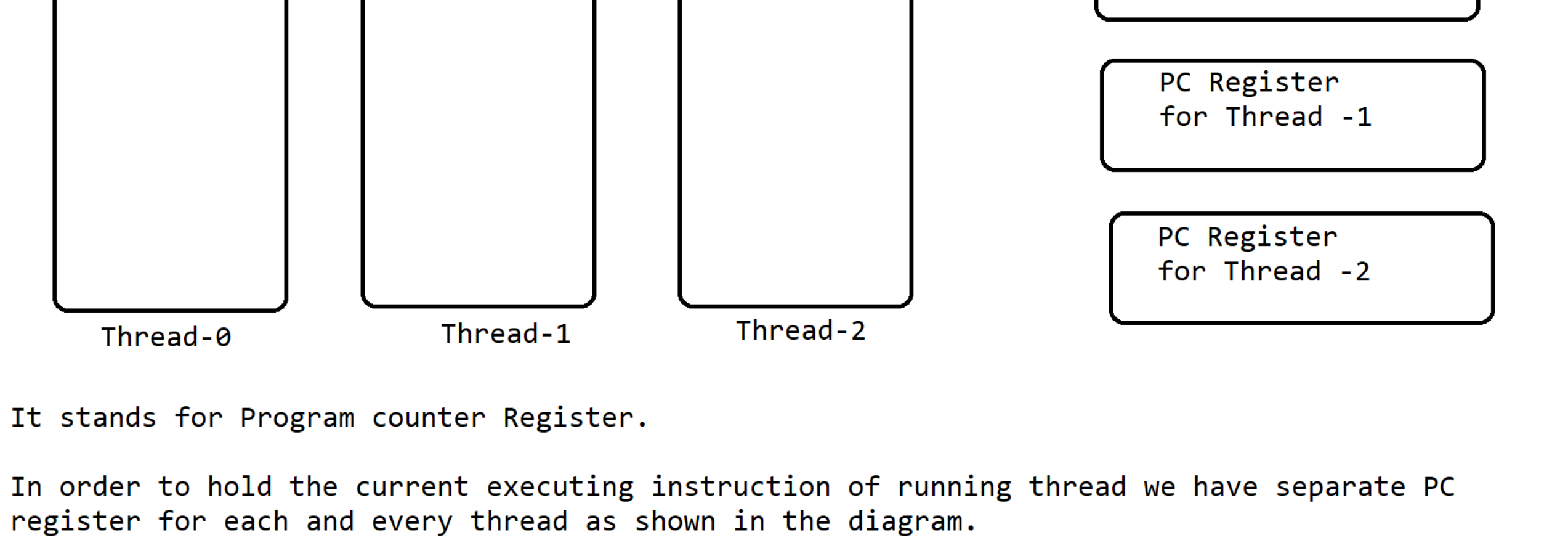
- 1) Local Variable arrays
- 2) Frame Data
- 3) Operand Stack.

We have multiple stack area for a single JVM.

Everytime we create a new thread in java then JVM will create a separate Runtime Stack Memory.[Multithreading]

PC Register :

* It stands for Program Counter Register.



It stands for Program counter Register.

In order to hold the current executing instruction of running thread we have separate PC register for each and every thread as shown in the diagram.

Native Method Stack

Native method means, the java methods which are written by using native languages like C and C++.

In order to write native method we need native method library support.

Native method stack will hold the native method information in a separate stack.

Execution Engine : [Interpreter + JIT Compiler]

Interpreter

In java, JVM contains an interpreter which executes the program line by line.

Interpreter is slow in nature because at the time of execution if we make a mistake at line number 9 then it will throw the exception at line number 9 and after solving the exception again it will start the execution from line number 1 so it is slow in execution that is the reason to boost up the execution java software people has provided JIT compiler.

JIT Compiler : [Will convert bytecode to native code]

It stands for just in time compiler. The main purpose of JIT compiler to boost up the execution so the execution of the program will be completed as soon as possible.

JIT compiler holds the repeated instruction like method signature, variables, native method code and make it available to JVM at the time of execution so the overall execution becomes very fast.

HAS-A Relation : [A class as a proptry]

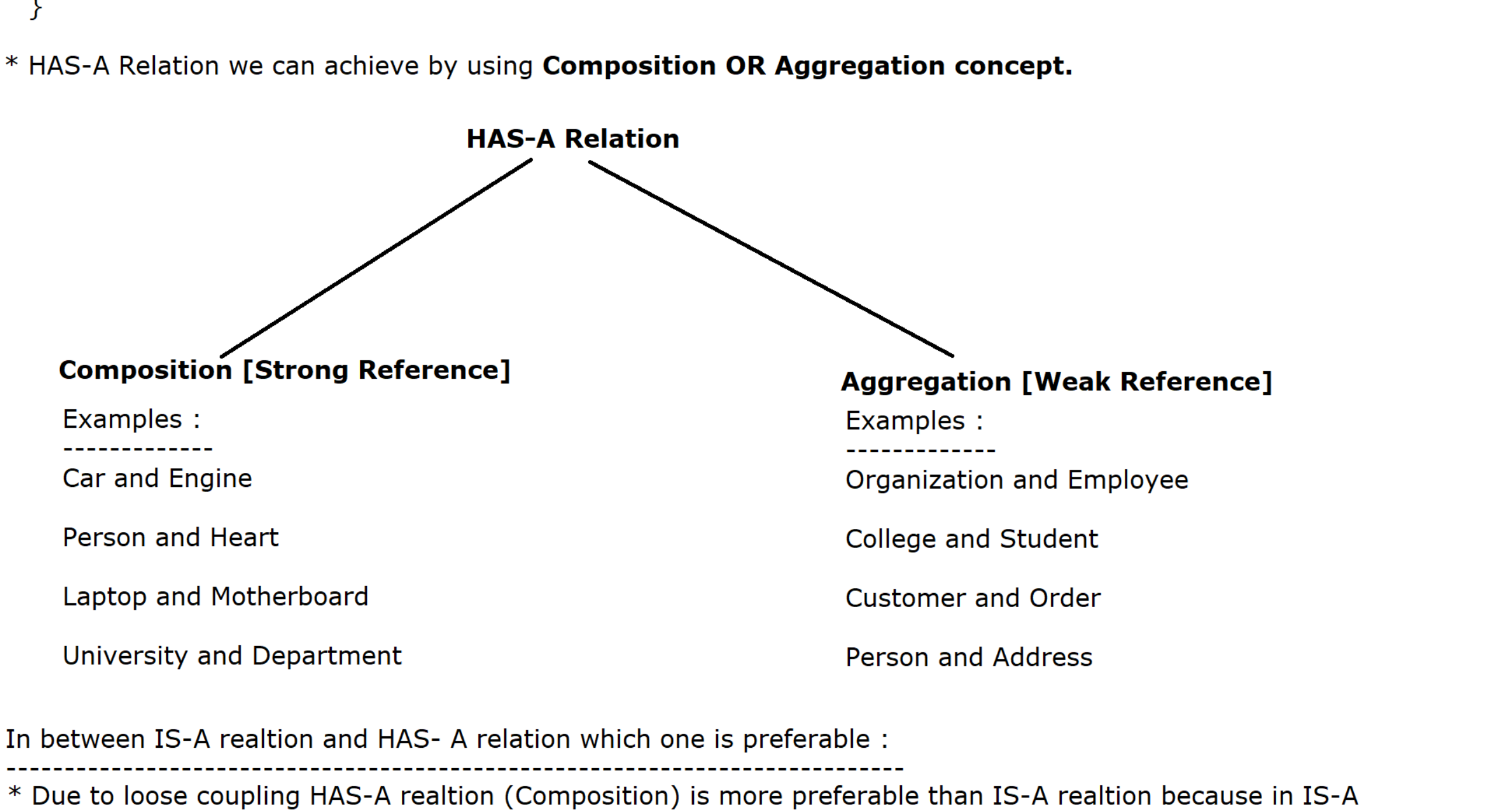
* If we use a class as a **property to another class** then It is called HAS-A Relation.

Example :

```
class Engine
{
}

class Car
{
    private Engine engine; //HAS-A Relation [Engine class we are using as a proptry to Car class]
}
```

* HAS-A Relation we can achieve by using **Composition OR Aggregation concept.**



In between IS-A realtion and HAS- A relation which one is preferable :

* Due to loose coupling HAS-A realtion (Composition) is more preferable than IS-A realtion because in IS-A realtion, If we modify the super class content then automatically sub class content will also modify where as on the other hand HAS- A relation provides accessibility features.